

PRAKTIKUM MACHINE LEARNING

Bab 8

Support Vector Machine

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

2 Oktober 2026

Daftar Isi

Support Vector Machine	3
Ringkasan Konsep	4
8.11 Latihan Praktikum (Modul Bab 8)	4
Percobaan Mandiri	10
Kesimpulan Bab 8	19

Tools yang Digunakan

Python 3.12.3 • numpy 2.5.3 • pandas 3.0.6 • matplotlib 3.11.2 • seaborn 0.13.2 • scikit-learn 1.9.1 • imbalanced-learn 0.14.2

Sel 0 - Persiapan Awal (Khusus Google Colab)

Kalau notebook ini dibuka di Google Colab, jalankan cell di bawah ini dulu untuk memastikan semua paket yang dipakai sudah tersedia. Kalau dijalankan di Jupyter lokal, cell ini tidak mengubah apa-apa.

In [1] - Kode Program

```
1 # Sel 0: cek paket (sudah tersedia di venv / Colab)
2 import sklearn, pandas, numpy, matplotlib
3 print("Paket siap.")
```

Paket siap.

Praktikum Machine Learning: Bab 8

Support Vector Machine

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini saya jalankan di Google Colab / Jupyter Notebook. Bab 8 ini mengerjakan **Latihan Praktikum Modul Bab 8 bagian 8.11**, memakai dataset **Breast Cancer** dari `sklearn.datasets.load_breast_cancer`. Tidak perlu file CSV apa pun.

Ringkasan Konsep

- **Margin maksimum dan support vector:** SVM mencari hyperplane (garis pemisah) yang memisahkan dua kelas dengan jarak sejauh mungkin. Titik-titik data terdekat yang menentukan posisi garis itu disebut support vector.
- **Kernel trick:** trik untuk menangani data yang tidak bisa dipisah garis lurus, datanya dipetakan ke ruang berdimensi lebih tinggi. Kernel yang umum dipakai: linear, polynomial, dan RBF.
- **Parameter C:** mengatur trade-off antara margin yang lebar dan kesalahan klasifikasi. C kecil mentolerir beberapa salah klasifikasi demi margin lebar, C besar memaksa model mengikuti data training.
- **Parameter gamma:** mengatur seberapa jauh pengaruh satu titik training. Gamma kecil membuat pengaruhnya melebar (boundary halus), gamma besar membuat pengaruhnya sempit (boundary berlekuk).
- **Scaling wajib:** SVM sensitif terhadap skala fitur karena dia menghitung jarak, jadi fitur harus distandarisasi dulu (misalnya pakai StandardScaler) sebelum training.

8.11 Latihan Praktikum (Modul Bab 8)

Di bagian ini saya mengerjakan dua Latihan Praktikum dari modul Bab 8 bagian 8.11, yaitu perbandingan kernel SVM dan visualisasi decision boundary.

Praktikum 1 - Perbandingan Kernel

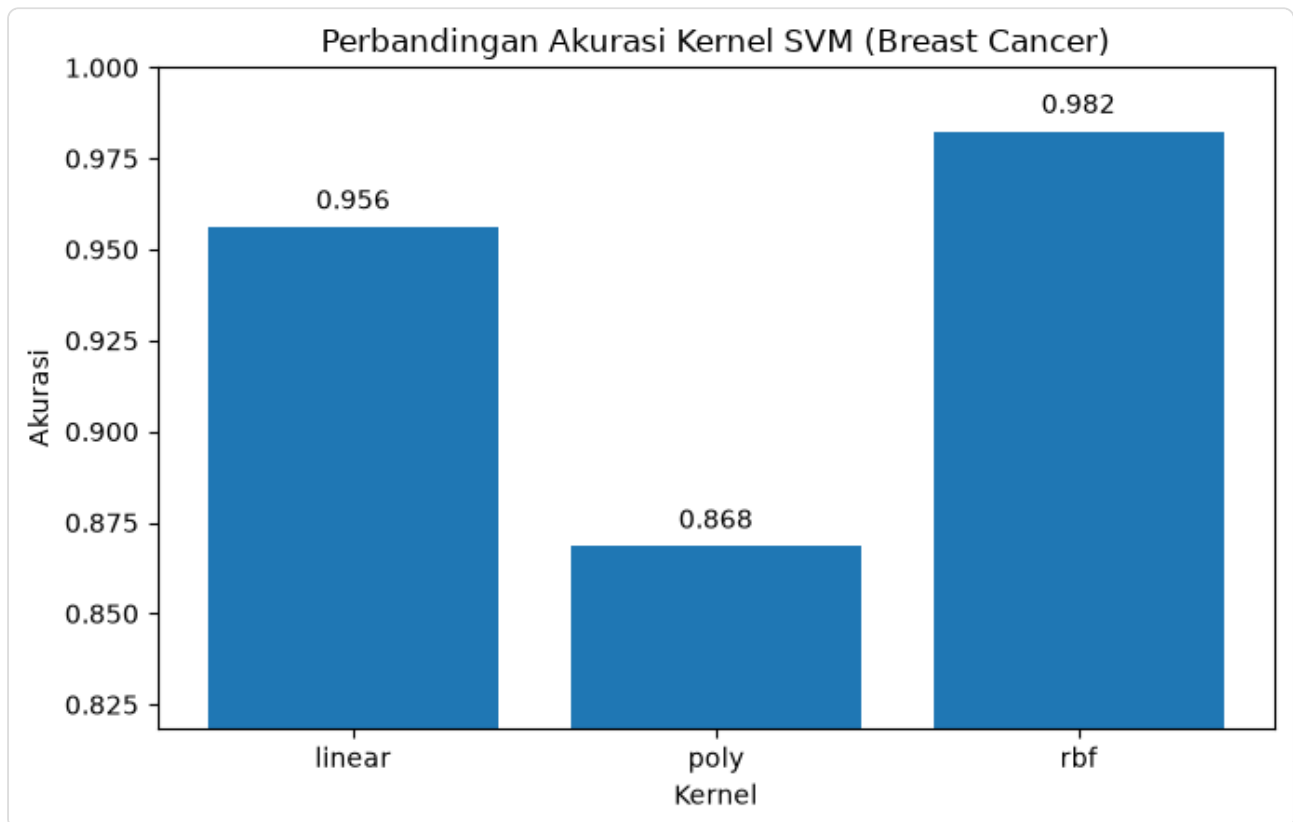
Tujuan: membandingkan performa kernel linear, polynomial, dan RBF pada dataset yang sama.

Bagian ini saya mengerjakan Praktikum 8.7 dari modul. Saya memakai dataset Breast Cancer yang dibagi 80/20 (`random_state=42`), lalu tiap kernel dilatih lewat Pipeline berisi `StandardScaler` dan `SVC`. Di akhir saya tambahkan grafik batang untuk membandingkan akurasi.

In [2] - Kode Program

```
1 from sklearn.datasets import load_breast_cancer
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.pipeline import Pipeline
5 from sklearn.svm import SVC
6 from sklearn.metrics import accuracy_score
7 import matplotlib.pyplot as plt
8
9 X, y = load_breast_cancer(return_X_y=True)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
11 random_state=42)
12
13 hasil = {}
14 for kernel in ["linear", "poly", "rbf"]:
15     model = Pipeline([("scaler", StandardScaler()), ("svm",
16 SVC(kernel=kernel))])
17     model.fit(X_train, y_train)
18     acc = accuracy_score(y_test, model.predict(X_test))
19     print(f"Kernel {kernel}: akurasi = {acc:.3f}")
20     hasil[kernel] = acc
21
22 # Grafik batang perbandingan akurasi
23 plt.figure(figsize=(7, 4.5))
24 plt.bar(hasil.keys(), hasil.values())
25 plt.title("Perbandingan Akurasi Kernel SVM (Breast Cancer)")
26 plt.xlabel("Kernel")
27 plt.ylabel("Akurasi")
28 plt.ylim(min(hasil.values()) - 0.05, 1.0)
29 for k, v in hasil.items():
30     plt.text(k, v + 0.005, f"{v:.3f}", ha="center")
31 plt.tight_layout()
32 plt.show()
```

```
Kernel linear: akurasi = 0.956
Kernel poly: akurasi = 0.868
Kernel rbf: akurasi = 0.982
```



Gambar 1

Penjelasan outputnya: - Saya memakai dataset Breast Cancer (569 baris, 30 fitur) yang dibagi 80/20 dengan `random_state=42`, jadi data test-nya 114 baris. - Hasilnya: kernel linear akurasinya 0.956, polynomial 0.868, dan RBF 0.982. - Kernel RBF paling tinggi karena dia bisa membuat batas keputusan yang melengkung mengikuti pola data, sedangkan linear cuma bisa garis lurus. - Kernel polynomial malah paling rendah di sini, kemungkinan bentuk polinomial bawaannya kurang cocok dengan pola dataset ini. - Semua model dilatih lewat Pipeline yang diawali `StandardScaler`, jadi scaling dihitung dari data training saja dan tidak bocor ke data test.

Praktikum 2 - Visualisasi Decision Boundary 2 Fitur

Tujuan: memvisualisasikan pengaruh kernel dan parameter C/γ pada decision boundary.

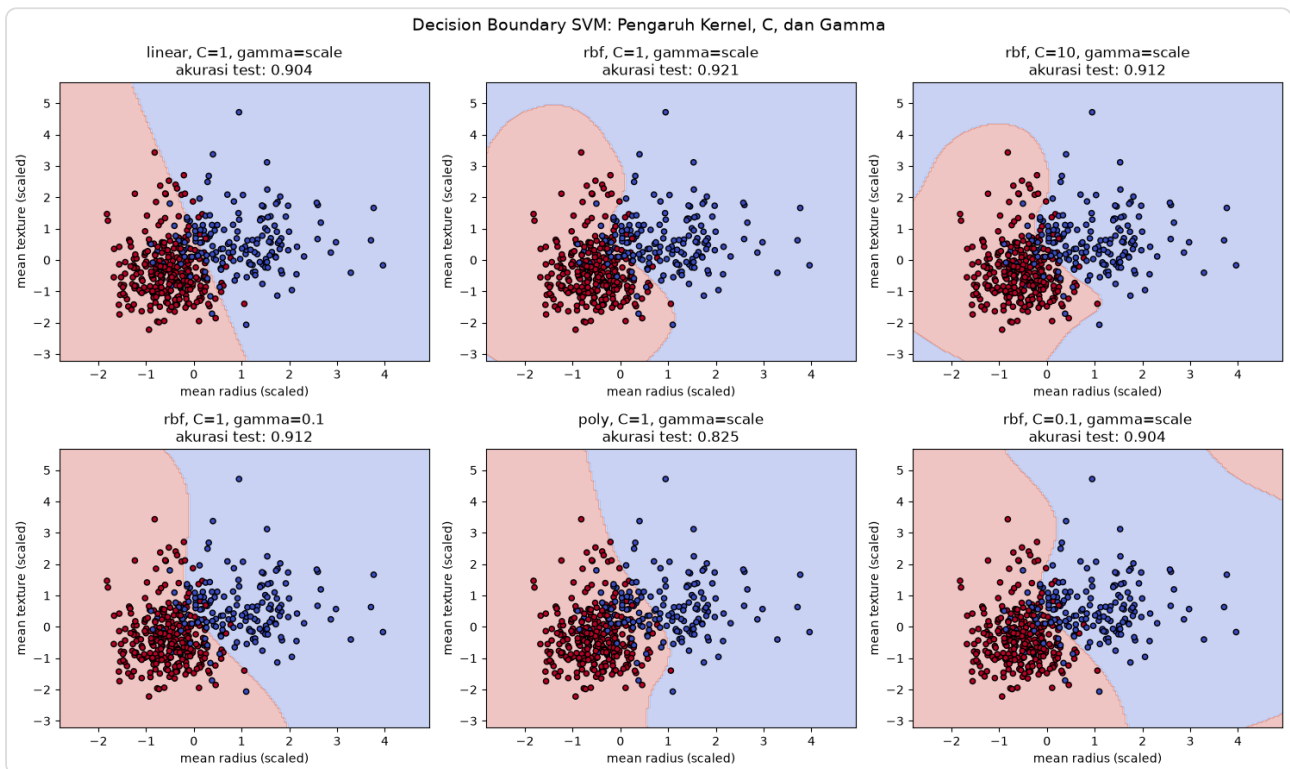
Instruksi: pilih 2 fitur dari dataset `load_breast_cancer` (saya pakai `mean_radius` dan `mean_texture`, yaitu kolom 0 dan 1), latih SVM dengan beberapa kombinasi kernel/ C/γ , dan visualisasikan decision boundary masing-masing dalam subplot berdampingan.

In [3] - Kode Program

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.svm import SVC
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7
8 X2 = X[:, [0, 1]] # mean radius, mean texture
9 X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y, test_size=0.2,
random_state=42)
10
11 scaler2 = StandardScaler()
12 X2_train_s = scaler2.fit_transform(X2_train)
13 X2_test_s = scaler2.transform(X2_test)
14
15 kombinasi = [
16     ("linear", 1, "scale"),
17     ("rbf", 1, "scale"),
18     ("rbf", 10, "scale"),
19     ("rbf", 1, 0.1),
20     ("poly", 1, "scale"),
21     ("rbf", 0.1, "scale"),
22 ]
23
24 fig, axes = plt.subplots(2, 3, figsize=(15, 9))
25 for ax, (kernel, C, gamma) in zip(axes.flat, kombinasi):
26     clf = SVC(kernel=kernel, C=C, gamma=gamma, random_state=42)
27     clf.fit(X2_train_s, y2_train)
28     x_min, x_max = X2_train_s[:, 0].min() - 1, X2_train_s[:, 0].max() + 1
29     y_min, y_max = X2_train_s[:, 1].min() - 1, X2_train_s[:, 1].max() + 1
30     xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
31                          np.arange(y_min, y_max, 0.05))
32     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
33     ax.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
34     ax.scatter(X2_train_s[:, 0], X2_train_s[:, 1], c=y2_train, cmap="coolwarm",
edgecolors="k", s=20)
35     acc2 = accuracy_score(y2_test, clf.predict(X2_test_s))
36     ax.set_title(f"{kernel}, C={C}, gamma={gamma} \nakurasi test: {acc2:.3f}")
37     ax.set_xlabel("mean radius (scaled)")
38     ax.set_ylabel("mean texture (scaled)")
39
40 fig.suptitle("Decision Boundary SVM: Pengaruh Kernel, C, dan Gamma", fontsize=14)
41 plt.tight_layout()
42 plt.show()

```



Gambar 2

Penjelasan outputnya: - Saya hanya memakai 2 fitur (mean radius dan mean texture) supaya decision boundary-nya bisa digambar, fiturnya di-scale dulu baru datanya dibagi 80/20. - linear, $C=1$ (akurasi 0.904): batasnya garis lurus, rapi tapi kaku, beberapa titik di dekat garis jadi salah kelas. - rbf, $C=1$ (akurasi 0.921): batasnya melengkung mengikuti sebaran titik, akurasinya paling tinggi di antara enam kombinasi. - rbf, $C=10$ (akurasi 0.912): C besar membuat margin makin ketat dan batasnya makin berlekuk mengikuti titik training, akurasinya sedikit turun dibanding $C=1$. - rbf, $\gamma=0.1$ (akurasi 0.912): γ kecil membuat pengaruh tiap titik melebar sehingga batasnya jadi lebih halus. - poly, $C=1$ (akurasi 0.825): akurasinya paling rendah, bentuk boundary polinomialnya kurang cocok untuk dua fitur ini. - rbf, $C=0.1$ (akurasi 0.904): C kecil membuat margin longgar dan modelnya lebih sederhana dengan batas yang halus.

Percobaan Mandiri

Tiga percobaan tambahan di luar Latihan Praktikum untuk melihat lebih jauh perilaku SVM: pengaruh parameter C terhadap margin, kemampuan kernel di data non-linear, dan pentingnya scaling fitur.

Percobaan Mandiri 1 - C Kecil vs C Besar pada SVM Linear

In [4] - Kode Program

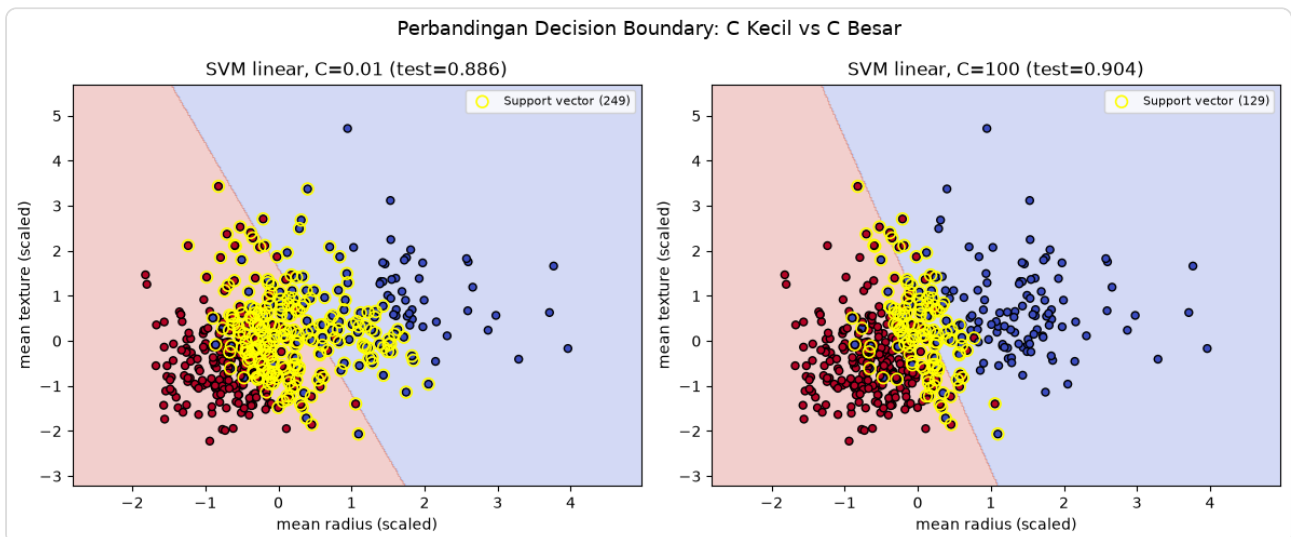
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.svm import SVC
7 from sklearn.metrics import accuracy_score
8
9 X, y = load_breast_cancer(return_X_y=True)
10 X2 = X[:, [0, 1]] # mean radius dan mean texture
11 X_train, X_test, y_train, y_test = train_test_split(X2, y, test_size=0.2,
random_state=42)
12
13 scaler = StandardScaler()
14 X_train_s = scaler.fit_transform(X_train)
15 X_test_s = scaler.transform(X_test)
16
17 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
18 for ax, C in zip(axes, [0.01, 100]):
19     clf = SVC(kernel="linear", C=C)
20     clf.fit(X_train_s, y_train)
21     acc_train = accuracy_score(y_train, clf.predict(X_train_s))
22     acc_test = accuracy_score(y_test, clf.predict(X_test_s))
23     n_sv = sum(clf.n_support_)
24     print(f"C={C}: train={acc_train:.3f}, test={acc_test:.3f}, support
vector={n_sv}")
25
26     x_min, x_max = X_train_s[:, 0].min() - 1, X_train_s[:, 0].max() + 1
27     y_min, y_max = X_train_s[:, 1].min() - 1, X_train_s[:, 1].max() + 1
28     xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
29                           np.arange(y_min, y_max, 0.02))
30     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
31     ax.contourf(xx, yy, Z, alpha=0.25, cmap=plt.cm.coolwarm)
32     ax.scatter(X_train_s[:, 0], X_train_s[:, 1], c=y_train,
33               cmap=plt.cm.coolwarm, edgecolor="k", s=25)
34     ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
35               facecolor="none", edgecolor="yellow", s=60, linewidth=1.2,
36               label=f"Support vector ({n_sv})")
37     ax.set_title(f"SVM linear, C={C} (test={acc_test:.3f})")
38     ax.set_xlabel("mean radius (scaled)")
39     ax.set_ylabel("mean texture (scaled)")
40     ax.legend(loc="upper right", fontsize=8)
41 plt.suptitle("Perbandingan Decision Boundary: C Kecil vs C Besar", fontsize=13)
42 plt.tight_layout()
43 plt.show()

```

C=0.01: train=0.862, test=0.886, support vector=249

C=100: train=0.888, test=0.904, support vector=129



Gambar 3

Penjelasan outputnya: - Saya hanya memakai 2 fitur (mean radius dan mean texture) yang sudah di-scale supaya boundary-nya bisa digambar. - C=0.01: akurasi train 0.862, test 0.886, support vector-nya 249. Batasnya lurus dan marginnya lebar, modelnya rela salah di beberapa titik training. - C=100: akurasi train 0.888, test 0.904, support vector-nya cuma 129. Batasnya lebih ketat mengikuti titik-titik dan marginnya lebih sempit. - Kesimpulan: C kecil membuat margin yang lebar dan lunak (support vector lebih banyak), sedangkan C besar membuat margin yang keras dan ketat mengikuti data.

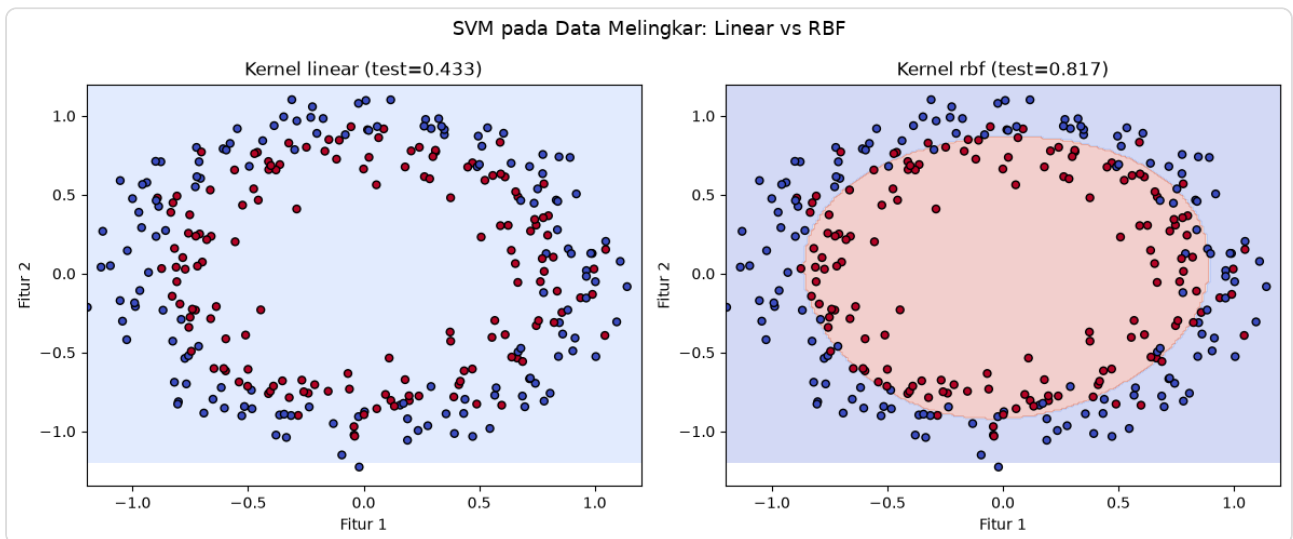
Percobaan Mandiri 2 - Kernel Linear vs RBF di Data Non-Linear

In [5] - Kode Program

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_circles
4 from sklearn.model_selection import train_test_split
5 from sklearn.svm import SVC
6 from sklearn.metrics import accuracy_score
7
8 X, y = make_circles(n_samples=300, noise=0.1, random_state=42)
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
10
11 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
12 for ax, kernel in zip(axes, ["linear", "rbf"]):
13     clf = SVC(kernel=kernel)
14     clf.fit(X_train, y_train)
15     acc = accuracy_score(y_test, clf.predict(X_test))
16     print(f"Kernel {kernel}: akurasi test = {acc:.3f}")
17
18     xx, yy = np.meshgrid(np.linspace(-1.2, 1.2, 200),
19                           np.linspace(-1.2, 1.2, 200))
20     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
21     ax.contourf(xx, yy, Z, alpha=0.25, cmap=plt.cm.coolwarm)
22     ax.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, edgecolor="k", s=25)
23     ax.set_title(f"Kernel {kernel} (test={acc:.3f})")
24     ax.set_xlabel("Fitur 1")
25     ax.set_ylabel("Fitur 2")
26 plt.suptitle("SVM pada Data Melingkar: Linear vs RBF", fontsize=13)
27 plt.tight_layout()
28 plt.show()
```

Kernel linear: akurasi test = 0.433

Kernel rbf: akurasi test = 0.817



Gambar 4

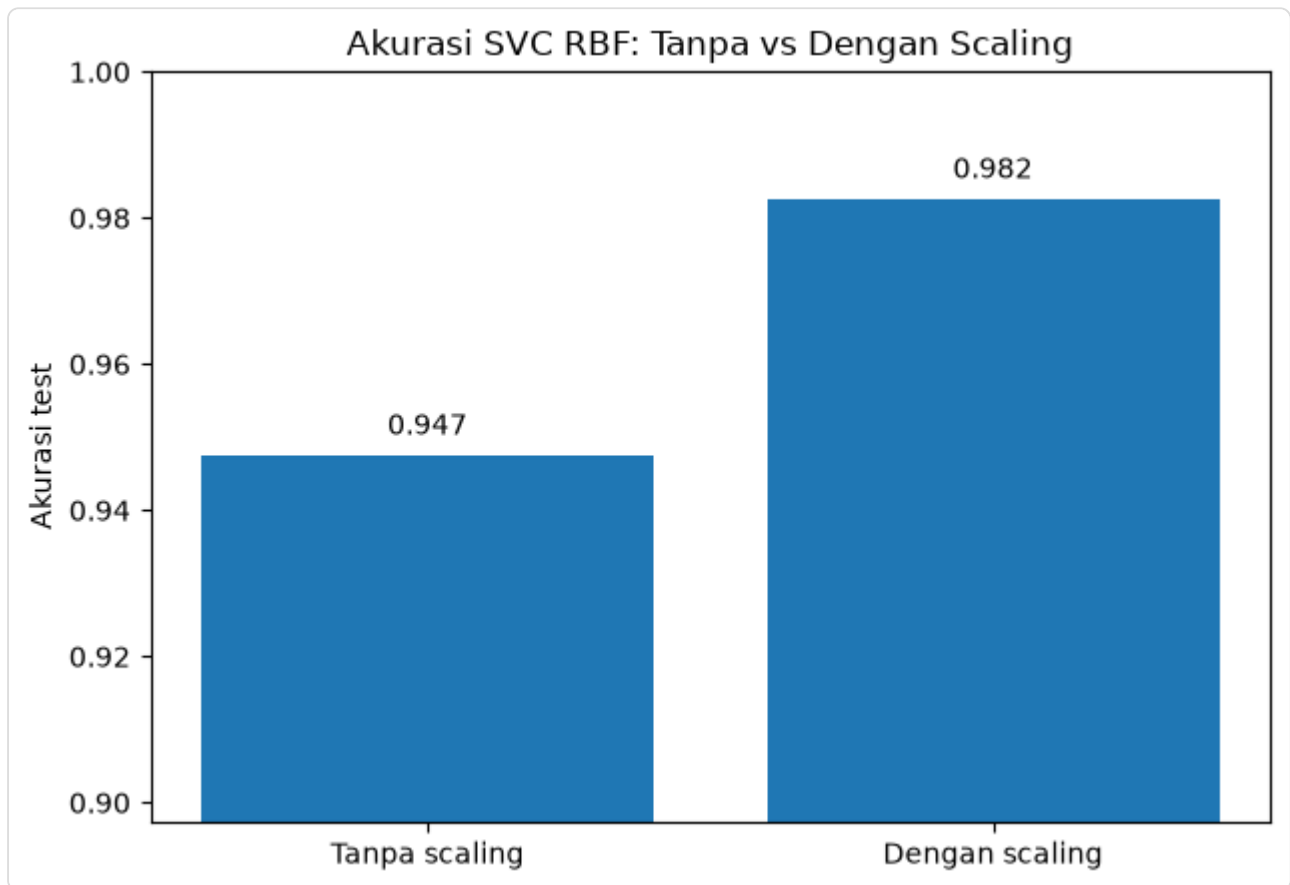
Penjelasan outputnya: - Saya membuat data melingkar dengan `make_circles` (300 sampel) yang memang tidak bisa dipisah dengan garis lurus. - Kernel linear: akurasi test cuma 0.433, bahkan lebih buruk dari tebakan acak. Dari grafiknya kelihatan garis lurusnya gagal memisahkan lingkaran dalam dari lingkaran luar. - Kernel RBF: akurasi test 0.817, batasnya melingkar mengikuti bentuk data. - Kesimpulan: kernel linear memang tidak cocok untuk data non-linear seperti ini, sedangkan kernel RBF bisa membuat batas yang melengkung.

Percobaan Mandiri 3 - Scaling vs Tanpa Scaling

In [6] - Kode Program

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.svm import SVC
6 from sklearn.metrics import accuracy_score
7
8 X, y = load_breast_cancer(return_X_y=True)
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
10
11 hasil = {}
12 clf_tanpa = SVC(kernel="rbf")
13 clf_tanpa.fit(X_train, y_train)
14 hasil["Tanpa scaling"] = accuracy_score(y_test, clf_tanpa.predict(X_test))
15
16 scaler = StandardScaler()
17 X_train_s = scaler.fit_transform(X_train)
18 X_test_s = scaler.transform(X_test)
19 clf_dengan = SVC(kernel="rbf")
20 clf_dengan.fit(X_train_s, y_train)
21 hasil["Dengan scaling"] = accuracy_score(y_test, clf_dengan.predict(X_test_s))
22
23 for nama, acc in hasil.items():
24     print(f"{nama}: akurasi test = {acc:.3f}")
25
26 plt.figure(figsize=(6.5, 4.5))
27 plt.bar(hasil.keys(), hasil.values())
28 plt.title("Akurasi SVC RBF: Tanpa vs Dengan Scaling")
29 plt.ylabel("Akurasi test")
30 plt.ylim(min(hasil.values()) - 0.05, 1.0)
31 for k, v in hasil.items():
32     plt.text(k, v + 0.003, f"{v:.3f}", ha="center")
33 plt.tight_layout()
34 plt.show()
```

Tanpa scaling: akurasi test = 0.947
Dengan scaling: akurasi test = 0.982



Gambar 5

Penjelasan outputnya: - SVC dengan kernel RBF tanpa scaling: akurasi test 0.947. - SVC dengan kernel RBF memakai StandardScaler: akurasi test 0.982, naik 0.035. - Kesimpulan: SVM sangat sensitif terhadap skala fitur karena dia bekerja dengan menghitung jarak antar titik. Jadi scaling itu wajib sebelum melatih SVM.

Kesimpulan Bab 8

- Kernel RBF memberi akurasi tertinggi (0.982) pada dataset Breast Cancer full, mengalahkan linear (0.956) dan polynomial (0.868).
- Parameter C mengatur seberapa ketat model mengikuti data training: C besar membuat boundary berlekuk, C kecil membuat boundary halus.
- Parameter gamma mengatur jangkauan pengaruh tiap titik: gamma kecil membuat boundary halus, gamma besar membuat boundary mengikuti titik-titik individual.
- Scaling fitur itu wajib sebelum melatih SVM, dan visualisasi 2 fitur membuktikan pilihan kernel sangat menentukan bentuk decision boundary.